Atividade 4

Bruno Dalcantoni Cozac#13686323

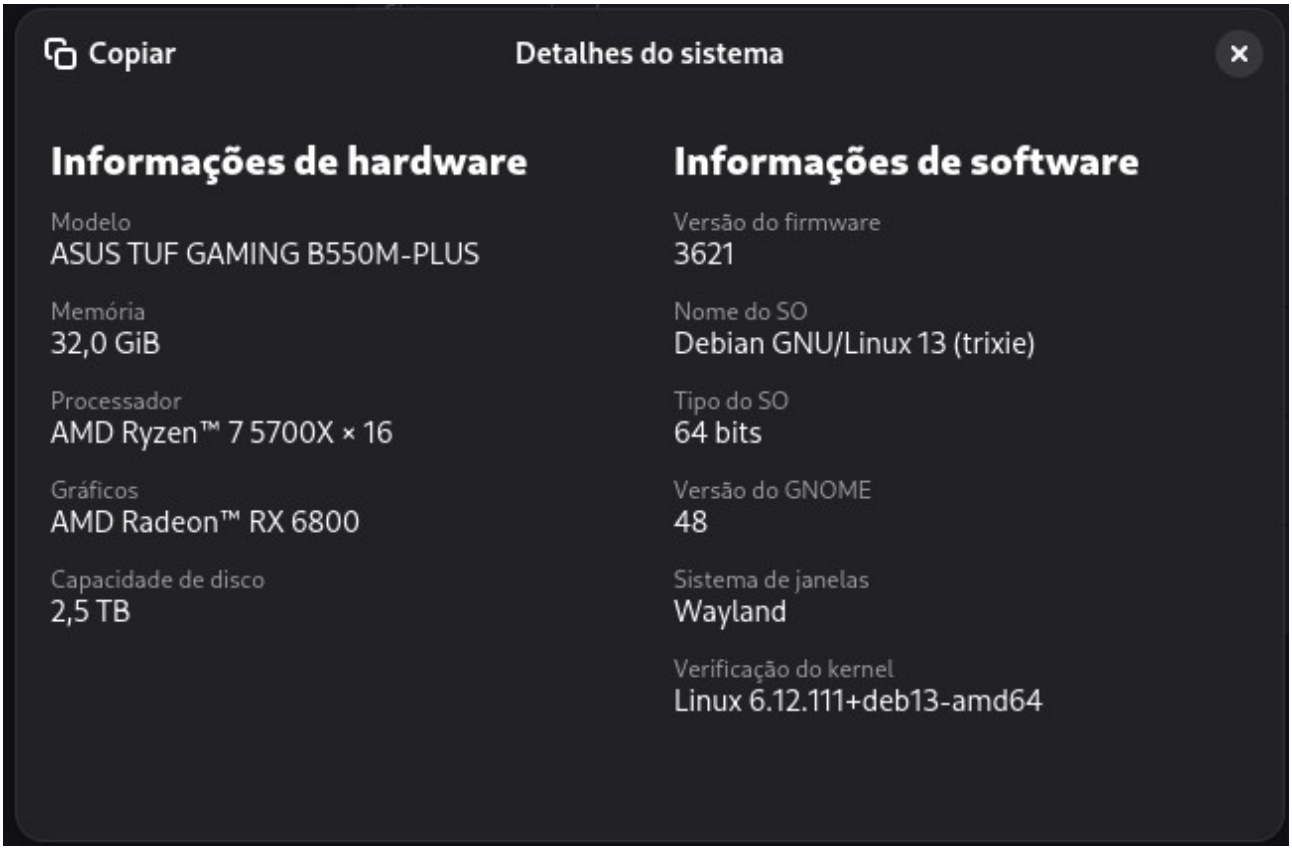
Fernando alee swaiden#12680836

Escolha de arquitetura:

Todos os testes foram realizados num computador com CPU AMD:

```
bruno@brunao-desktop:~$ lscpu
Arquitetura:                         x86_64
  Modo(s) operacional da CPU:        32-bit, 64-bit
  Tamanhos de endereço:              48 bits physical, 48 bits virtual
  Ordem dos bytes:                   Little Endian
CPU(s):                              16
  Lista de CPU(s) on-line:           0-15
ID de fornecedor:                    AuthenticAMD
  Nome do modelo:                    AMD Ryzen 7 5700X 8-Core Processor
    Família da CPU:                  25
    Modelo:                          33
    Thread(s) per núcleo:            2
    Núcleo(s) por soquete:           8
    Soquete(s):                      1
    Step:                            2
    Aumento de frequência:           desabilitado
    CPU(s) MHz de escala:            42%
    CPU MHz máx.:                    4935,0000
    CPU MHz mín.:                    550,0000
```

Embora não seja uma informação tão relevante quanto a configuração da CPU, segue também demais informações do sistema:



Escolha dos programas testados:

Todos os códigos foram disponibilizados no site:
https://benchmarksgame-team.pages.debian.net/benchmarksgame/

Para ter uma diversidade de situações na análise, selecionamos um programa com escrita de dados em disco(Mandelbrot) e outro sem escrita de dados(spectral-norm), ambos em linguagem C;

A principio, estima-se que o Mandelbrot não tenha um “Speed-up” significativo, já que as otimizações realizadas pelo GCC não interferem no gargalo causado pela escrita em disco.

OBS: No código do programa Spectral-norm havia uma função definida como inline que causava erro na compilação o0(este nível de otimização não aplica inline), sendo assim retiramos a definição do inline, sem alteração no resultado do algoritmo.

Resultados:

Ambos os códigos foram testados utilizando os níveis de otimização o0,o1,o2,o3,o3 com instruções específicas do processador(-march=native) e os.

Para obtenção dos dados de compilação foi utilizado o comando -ftime-report, para obtenção dos dados de execução, utilizamos o comando time e repetimos 5 vezes cada.

	Mandelbrot					
	o0	o1	o2	o3	o3 instruções os	os
Compilação	0,05	0,07	0,09	0,09	0,09	0,08
Tamanho(kb)	3719	4196	4518	4651	4803	4159
Execução	1,116	0,288	0,306	0,179	0,139	0,342
	1,118	0,289	0,312	0,173	0,139	0,347
	1,104	0,286	0,37	0,206	0,133	0,331
	1,098	0,286	0,287	0,173	0,112	0,354
	1,1	0,288	0,284	0,172	0,139	0,381
média	1,1072	0,2874	0,3118	0,1806	0,1324	0,351
mediana	1,104	0,288	0,306	0,173	0,139	0,347

Como esperado, o tempo de compilação e tamanho do arquivo aumentam conforme avançamos para otimizações mais agressivas, da mesma forma que o tempo de execução tende a diminuir.
A flag os teve desempenho entre o0 e o1, porém com tempo de compilação maior.

spectral norm

	o0	o1	o2	o3	o3 instruções os	os
Compilação	0,08	0,1	0,12	0,12	0,12	0,1
Tamanho(kb)	5184	5559	6226	6441	7027	5615
Execução	0,411	0,17	0,213	0,169	0,17	0,331
	0,409	0,2225	0,169	0,17	0,171	0,224
	0,44	0,169	0,169	0,171	0,171	0,226
	0,443	0,165	0,17	0,17	0,178	0,232
	0,423	0,166	0,169	0,17	0,173	0,228
média	0,4252	0,1785	0,178	0,17	0,1726	0,2482
mediana	0,423	0,169	0,169	0,17	0,171	0,228

Diferentemente do algoritmo anterior, não tivemos melhoria no tempo de execução, houve uma estagnação em o1, embora o tempo de compilação e tamanho do arquivo tenham aumentado. A flag os manteve desempenho entre o0 e 01, porém sem redução no tamanho do arquivo que justifique seu uso.

Código disponível em https://github.com/Brunz40/SSC0951---Desenvolvimento-de-C-digo-Otimizado/tree/Sem-IA/ex4